EmbeddedShiksha

embeddedshiksha.com

FreeRTOS Quick Reference

The key APIs, concepts and interview gotchas on one desk-side reference, for debugging FreeRTOS firmware and preparing for embedded systems interviews.

Tasks & Scheduling | Queues · Semaphores · Mutexes | Timers · Events · Notifications | Config · Memory · ISR | Interview Q&A

Learn. Practice. Grow Together.

01 RTOS Basics & Tasks
02 States & Scheduler
03 Queues & Semaphores
04 Mutexes & Priority Inversion
05 Software Timers & Event Groups
06 Task Notifications & Buffers
07 Config & Memory
08 ISR Safety & Interview Q&A

01 What is an RTOS? · Tasks

Real-time guarantees and the building blocks of every FreeRTOS application

RTOS
Real-Time OS
Guarantees task execution within a deadline.

HARD REAL-TIME
Miss = system failure
e.g. airbag controller, pacemaker.

SOFT REAL-TIME
Miss = degraded performance
e.g. media player.

FreeRTOS in one line
Open-source, ~10 KB flash, and runs on ARM, RISC-V, AVR and many more architectures.

xTaskCreate create a task

```
BaseType_t xTaskCreate(
    TaskFunction_t  pvTaskCode,     // function pointer: void fn(void *)
    const char *    pcName,         // debug name, ≤ configMAX_TASK_NAME_LEN
    uint16_t        usStackDepth,   // stack in WORDS (not bytes!)
    void *          pvParameters,   // arg passed to task function
    UBaseType_t     uxPriority,     // 0=lowest … (configMAX_PRIORITIES-1)=highest
    TaskHandle_t *  pxCreatedTask   // OUT: handle, or NULL
);
```

Return value
Returns `pdPASS` on success, or `errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY` if the FreeRTOS heap is exhausted.

Essential task APIs

`vTaskDelete(xTask);`	`NULL` = delete self
`vTaskDelay(pdMS_TO_TICKS(ms));`	Relative delay: blocks for the given number of ticks from now
`vTaskDelayUntil(&xLast, period);`	**Periodic tasks, drift-free.** Wakes at an absolute time
`vTaskSuspend(xTask);`	`NULL` = suspend self
`vTaskResume(xTask);`	Resume; called from another task only
`xTaskResumeFromISR(xTask);`	Resume from an ISR
`vTaskPrioritySet(xTask, pri);`	Change priority at runtime
`uxTaskPriorityGet(xTask);`	Read current priority
`eTaskGetState(xTask);`	Returns `eRunning` / `eReady` / `eBlocked` / `eSuspended` / `eDeleted`

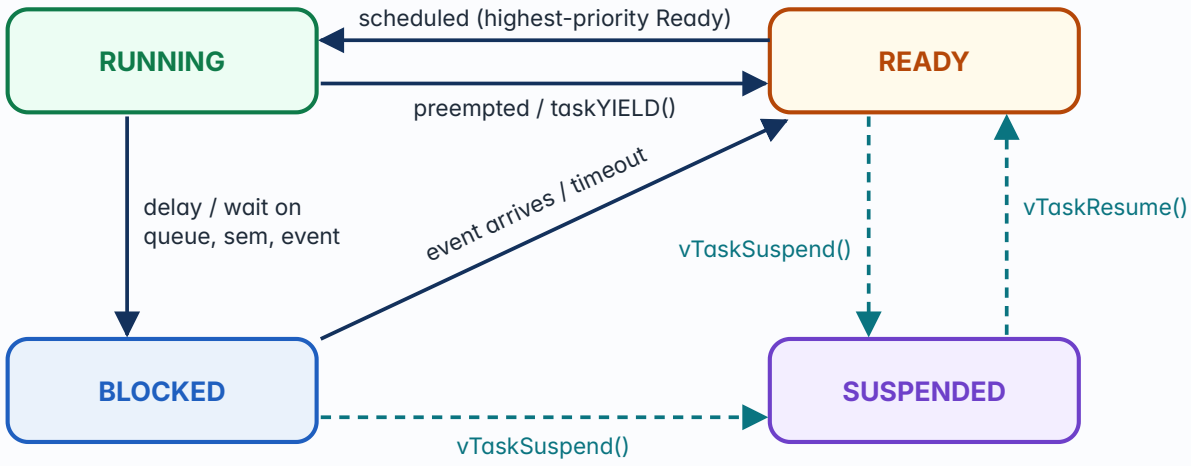
Task function template

```
void myTask(void *pvParams) {
    // cast params:  int val = *(int*)pvParams;
    TickType_t xLast = xTaskGetTickCount();
    for (;;) {
        // do work ...
        vTaskDelayUntil(&xLast, pdMS_TO_TICKS(10));   // 100 Hz
    }
    vTaskDelete(NULL);   // never reached (good practice)
}
```

02 Task State Machine · Scheduler

How tasks move between states and who runs next

Task state machine



Solid arrows: scheduler and blocking transitions. Dashed arrows: explicit suspend and resume. `vTaskSuspend()` works on a task in any state, including the running task itself (`vTaskSuspend(NULL)`). Only one task is RUNNING per core.

Scheduler

- **Preemptive (default):** a higher-priority task runs immediately when it becomes Ready.
- The tick interrupt fires every `1 / configTICK_RATE_HZ` seconds.
- `configTICK_RATE_HZ = 1000` → 1 ms tick resolution.
- **Idle task** (priority 0) is always Ready and never blocks.
- **Round-robin** within the same priority level (time-sliced; `configUSE_TIME_SLICING`, default 1).

Tick conversion
`pdMS_TO_TICKS(ms) = (ms × configTICK_RATE_HZ) / 1000`



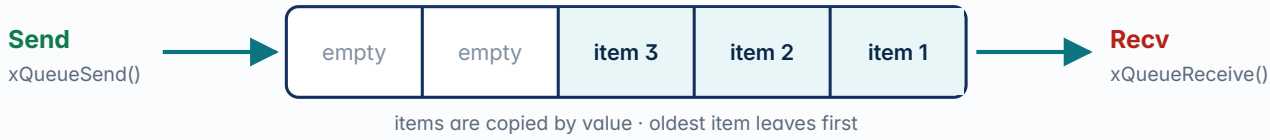
Preemptive scheduling: the highest-priority Ready task always gets the CPU.

03 Queues · Semaphores

Inter-task communication and synchronisation

Queues safe data passing between tasks

FIFO by default · thread-safe · blocks the sender when full and the receiver when empty.



items are copied by value · oldest item leaves first

CREATE & USE

`QueueHandle_t q = xQueueCreate(length, itemSize);`	`itemSize` in bytes
`xQueueSend(q, &item, ticksToWait);`	Same as `xQueueSendToBack()`
`xQueueSendToFront(q, &item, ticks);`	Insert at front
`xQueueReceive(q, &buf, ticksToWait);`	Removes item; returns `pdTRUE` / `pdFALSE`
`xQueuePeek(q, &buf, ticks);`	Reads without removing
`uxQueueMessagesWaiting(q);`	Items currently in queue
`uxQueueSpacesAvailable(q);`	Free slots

ISR VARIANTS — always use these from interrupts

```
xQueueSendFromISR(q, &item, &xHPTW);
xQueueReceiveFromISR(q, &buf, &xHPTW);
portYIELD_FROM_ISR(xHPTW);   // after ISR-safe calls, to yield immediately
```

`xHPTW` = `xHigherPriorityTaskWoken` (a `BaseType_t` initialised to `pdFALSE`).

Binary semaphore task ⇄ ISR signalling

- Value: **0** (unavailable) or **1** (available).
- Typical use: the **ISR gives** and the **task takes** (event-driven processing).

Counting semaphore resource counting

- Value: 0 ... `maxCount`. Counts available resources or events.
- Give: `count++` · Take: `count--` (blocks when count == 0).
- Use cases: track items in a buffer, limit concurrent access (e.g. 3 threads max).

`SemaphoreHandle_t s = xSemaphoreCreateBinary();`	Create binary semaphore (starts empty)
`xSemaphoreGive(s);`	Release (set to 1)
`xSemaphoreTake(s, portMAX_DELAY);`	Acquire (blocks if 0)
`xSemaphoreGiveFromISR(s, &xHPTW);`	**From interrupt**
`xSemaphoreTakeFromISR(s, &xHPTW);`	Rare; prefer Give from ISR
`xSemaphoreCreateCounting(maxCount, initialCount);`	Create counting semaphore

04 Mutexes · Priority Inversion

Mutual exclusion for shared resources

Mutex mutual exclusion: protect shared resources

- Like a binary semaphore, **but the task that Takes it must Give it back** (ownership).
- Built-in **priority inheritance** prevents unbounded priority inversion.

`SemaphoreHandle_t m = xSemaphoreCreateMutex();`	Create mutex
`xSemaphoreTake(m, portMAX_DELAY);`	**Lock**: enter critical section
`xSemaphoreGive(m);`	**Unlock**: exit critical section

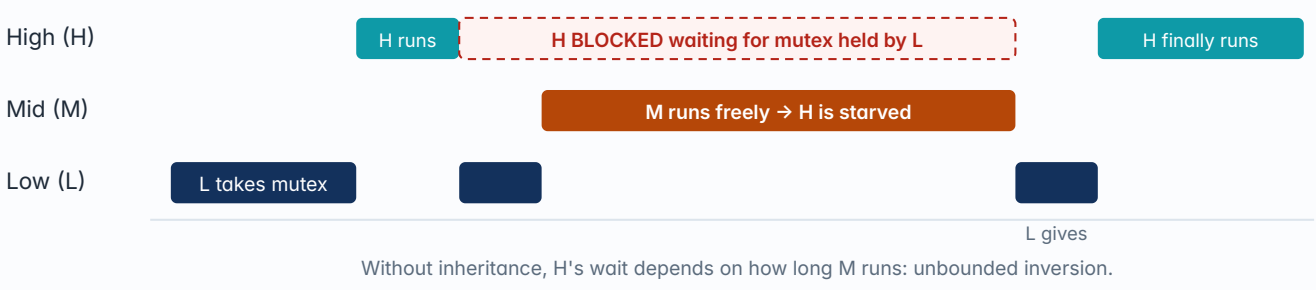
Never from an ISR

Never call `xSemaphoreGive` / `xSemaphoreTake` on a mutex from an ISR. Use a binary semaphore for ISR synchronisation.

RECURSIVE MUTEX: THE SAME TASK CAN LOCK MULTIPLE TIMES

```
xSemaphoreCreateRecursiveMutex();
xSemaphoreTakeRecursive(m, ticks);    // each Take needs a matching Give
xSemaphoreGiveRecursive(m);
```

Priority inversion



The problem
A low-priority task holds the mutex, so the high-priority task BLOCKS waiting for it. A mid-priority task then runs freely, and the high-priority task is starved.

FreeRTOS mutex fix: priority inheritance
The low-priority task temporarily inherits the high-priority task's priority until it releases the mutex. The mid-priority task can no longer preempt it.

Semaphore vs mutex quick diff

	Binary semaphore	Counting semaphore	Mutex
Ownership	No	No	**Yes (task)**
ISR-safe Give	**Yes**	**Yes**	No
Priority inheritance	No	No	**Yes ✓**

05 Software Timers · Event Groups

Advanced FreeRTOS synchronisation mechanisms

Software timers

- Run a callback after a timeout, with **no extra task needed**.
- Callbacks run in the **Timer Daemon Task** context (not in an ISR, not in your task).
- **One-shot:** fires once · **Auto-reload:** fires repeatedly.

Config required
`configUSE_TIMERS = 1`, and `configTIMER_TASK_STACK_DEPTH` must be set (along with `configTIMER_TASK_PRIORITY` and `configTIMER_QUEUE_LENGTH`).

CREATE & CONTROL

```
TimerHandle_t t = xTimerCreate(
    "name",                 // debug name
    pdMS_TO_TICKS(500),     // period
    pdTRUE,                 // pdTRUE=auto-reload, pdFALSE=one-shot
    (void*)timerID,         // timer ID (pvTimerGetTimerID)
    vTimerCallback          // void fn(TimerHandle_t)
);
```

`xTimerStart(t, 0);`	Start (or restart if already running)
`xTimerStop(t, 0);`	Stop
`xTimerReset(t, 0);`	Restart the period from now
`xTimerChangePeriod(t, ticks, 0);`	Change the period; also starts the timer if it is stopped
`pvTimerGetTimerID(t);`	Retrieve the ID stored at creation
`pcTimerGetName(t);`	Get the debug name

Keep callbacks short
Timer callbacks MUST be short and non-blocking. Never call `vTaskDelay` in one: it stalls the daemon task and every other timer.

Event groups multi-event synchronisation

- **24 usable event bits** (bits 0–23) per group on 32-bit tick ports.
- Tasks can wait for **ANY** or **ALL** of a set of bits, atomically.
- Great for synchronising N tasks at a rendezvous point.

`EventGroupHandle_t eg = xEventGroupCreate();`	Create
`xEventGroupSetBits(eg, BIT_0 | BIT_2);`	Set bits (from a task, or use the ISR variant)
`xEventGroupClearBits(eg, BIT_0);`	Clear bits
`xEventGroupGetBits(eg);`	Read current bits (non-blocking)

```
#define BIT_SENSOR  (1 << 0)
#define BIT_COMMS   (1 << 1)

xEventGroupWaitBits(
    eg,              // event group
    BIT_0 | BIT_1,   // bits to wait for
    pdTRUE,          // clear on exit?
    pdTRUE,          // wait for ALL? (pdFALSE = wait for ANY)
    portMAX_DELAY    // timeout
);
```

06 Task Notifications · Stream & Message Buffers

The fastest signalling path, plus byte and message streams

Task notifications fastest mechanism

DIRECT-TO-TASK
No intermediate object (queue or semaphore) is needed.

~45% FASTER
Faster and uses less RAM than unblocking a task with a binary semaphore.

PER TASK
One 32-bit notification value + one pending flag (per array entry on v10.4+).

Limitation
Only one task can wait (the receiver is always a specific task), and only one value is stored.

NOTIFICATION ACTIONS (eACTION)

`eNoAction`	Just set the pending flag; don't change the value
`eSetBits`	OR the value into the notification value (like an event group)
`eIncrement`	Increment the notification value (like a counting semaphore)
`eSetValueWithOverwrite`	Overwrite the value (like a queue of length 1, with overwrite)
`eSetValueWithoutOverwrite`	Write only if no notification is already pending

KEY APIS

`xTaskNotify(handle, value, eAction);`	Send from a task
`xTaskNotifyFromISR(handle, value, eAction, &xHPTW);`	Send from an ISR
`xTaskNotifyWait(clearEntry, clearExit, &val, ticks);`	Wait, then read the value
`xTaskNotifyGive(handle);`	Simplified: increment (like semaphore Give)
`vTaskNotifyGiveFromISR(handle, &xHPTW);`	Simplified Give from an ISR
`ulTaskNotifyTake(xClear, ticks);`	Simplified: decrement (like semaphore Take)

Stream & message buffers FreeRTOS 10+

STREAMBUFFER
Arbitrary byte stream
Like a pipe or FIFO of bytes.

MESSAGEBUFFER
Stream of discrete messages
Each message is stored with a length prefix.

`xStreamBufferCreate(sizeBytes, triggerLevel);`	Receiver unblocks once `triggerLevel` bytes are available
`xStreamBufferSend(sb, pData, len, ticks);`	Write bytes (`…FromISR` variant available)
`xStreamBufferReceive(sb, pBuf, len, ticks);`	Read bytes
`xMessageBufferCreate(sizeBytes);`	Create a message buffer
`xMessageBufferSend / xMessageBufferReceive`	Whole messages in and out

Single writer, single reader
Stream and message buffers assume one writer and one reader. With multiple writers or readers, protect access with a critical section or mutex.

07 FreeRTOSConfig.h · Memory Management

System setup and heap management

FreeRTOSConfig.h must-know settings

`configUSE_PREEMPTION`	`1`	1 = preemptive, 0 = cooperative
`configTICK_RATE_HZ`	`1000`	Tick frequency → 1 ms resolution
`configMAX_PRIORITIES`	`5`	Keep low (5–8) to save RAM
`configMINIMAL_STACK_SIZE`	`128`	Idle task stack (words, not bytes)
`configTOTAL_HEAP_SIZE`	`(16*1024)`	Total dynamic heap bytes
`configUSE_MUTEXES`	`1`	Enable mutex API
`configUSE_RECURSIVE_MUTEXES`	`1`	For nested locks
`configUSE_COUNTING_SEMAPHORES`	`1`	Enable counting semaphores
`configUSE_TIMERS`	`1`	Enable software timers
`configCHECK_FOR_STACK_OVERFLOW`	`2`	1 = basic, 2 = thorough (debug)
`configUSE_MALLOC_FAILED_HOOK`	`1`	Called when the heap is exhausted
`configUSE_IDLE_HOOK`	`1`	Idle task hook (low-power entry)
`configUSE_TICK_HOOK`	`1`	Tick interrupt callback

Memory management heap_1 ... heap_5

Scheme	Behaviour
heap_1	Simplest. `pvPortMalloc` only, NO free. No fragmentation. → Use when objects are created once at start-up and never deleted.
heap_2	Allows free(), but with no coalescing, so fragmentation is possible. → Legacy. Avoid for new projects.
heap_3	Wraps the standard `malloc` / `free` . A thread-safe wrapper only. → Use if the libc malloc is acceptable.
heap_4	First-fit with coalescing of adjacent free blocks. Most commonly used. RECOMMENDED → The right choice for most projects.
heap_5	The heap_4 algorithm across multiple non-contiguous memory regions (set up with `vPortDefineHeapRegions()`). → Use for MCUs with split SRAM / CCMRAM.

`pvPortMalloc(size);`	Allocates from the FreeRTOS heap
`vPortFree(ptr);`	Frees memory. Not supported by heap_1.
`vApplicationMallocFailedHook();`	Called when the heap is full
`vApplicationStackOverflowHook(task, name);`	Stack overflow hook

08 ISR Safety Rules · Interview Q&A

The key gotchas, and the questions interviewers ask most

ISR safety rules

Never
call API functions without the `FromISR` suffix inside an ISR.

Always
pass `&xHigherPriorityTaskWoken` (initialised to `pdFALSE`) to every FromISR function.

Always
end the ISR with `portYIELD_FROM_ISR(xHigherPriorityTaskWoken);`

Keep ISRs short
Defer the processing to a task: the ISR gives a semaphore and the task does the work.

ISR-safe functions FromISR variants

```
xQueueSendFromISR(q, &item, &xHPTW)
xQueueReceiveFromISR(q, &buf, &xHPTW)
xSemaphoreGiveFromISR(sem, &xHPTW)
xSemaphoreTakeFromISR(sem, &xHPTW)
xTaskNotifyFromISR(h, val, action, &xHPTW)
vTaskNotifyGiveFromISR(h, &xHPTW)
xEventGroupSetBitsFromISR(eg, bits, &xHPTW)
```

Top interview Q&A

Q What is the difference between a semaphore and a mutex?

A mutex has ownership: the same task that takes it must give it back. A mutex also has priority inheritance. A semaphore has no ownership, so any task or ISR can give it.

Q When do you use vTaskDelay vs vTaskDelayUntil?

`vTaskDelay` delays **from now**, so a periodic loop drifts over time. `vTaskDelayUntil` delays **until an absolute time**, giving drift-free periodic tasks.

Q Can you call vTaskDelay from an ISR?

No, never. Use a semaphore or task notification to unblock a task from the ISR.

Q What happens if two tasks have the same priority?

Round-robin scheduling: each gets one tick time-slice before switching.

Q heap_4 vs heap_5?

heap_5 is the heap_4 algorithm plus support for non-contiguous memory regions. For example, use heap_5 on an STM32 with both SRAM and CCMRAM.

Want to Go Beyond the Cheatsheet?

Quick reference guides are great for revision, but building strong embedded systems fundamentals requires consistent practice and structured preparation.

If you're preparing for embedded systems interviews or looking to strengthen your technical skills, explore EmbeddedShiksha's **Embedded System Interview Prep Bootcamp 6.0**.

Build confidence across key interview topics, practice important concepts, and take the next step in your embedded engineering journey.

Explore Bootcamp 6.0 →

www.embeddedshiksha.com

© EmbeddedShiksha · Learn. Practice. Grow Together. · FreeRTOS™ is a trademark of Amazon Web Services, Inc. This is an independent learning resource, not affiliated with or endorsed by the FreeRTOS project.